

資料探勘hw3

一、資料前處理

(一) 特徵值分析

本作業資料集包含員工權限申請紀錄，目標欄位 ACTION 表示是否授權（1 為核准，0 為未核准）。資料集存在明顯的類別不平衡問題，核准案例遠多於未核准案例，此特性在模型訓練時需特別處理（如設定 class_weight）。

輸入特徵共九個，均為類別型變數，各欄位說明如下：

欄位名稱	說明
RESOURCE	資源識別 ID
MGR_ID	員工主管 ID
ROLE_ROLLUP_1	公司角色群組分類 ID 1
ROLE_ROLLUP_2	公司角色群組分類 ID 2
ROLE_DEPTNAME	公司角色所屬部門 ID
ROLE_TITLE	公司職稱 ID
ROLE_FAMILY_DESC	角色家族延伸描述 ID
ROLE_FAMILY	角色家族 ID
ROLE_CODE	角色代碼 ID

由於所有欄位皆為類別型 ID，無法直接輸入模型，需透過編碼方式轉換為數值特徵。本作業採用兩種編碼方式：Target Encoding 與 One-hot Encoding，分別應用於不同版本的實驗中。

(二) Target Encoding (含 OOF 與 Smoothing)

Target encoding 的概念是以某個類別在訓練資料中對應的 target 平均值作為新的數值特徵。例如，若某個資源代碼在歷史資料中較常出現在授權案例中，則其 target encoded value 會較高。

然而若直接使用整份訓練資料的 ACTION 對同一份訓練資料做 target encoding，出現次數很少的類別可能會將自身的答案洩漏進特徵中，造成 data leakage。為降低此問題，本作業採用 Out-of-Fold (OOF) target encoding：訓練資料以 StratifiedKFold 切成多份，編碼某一 fold 時，只使用其他 folds 的資料建立 encoding mapping，再將結果填回該 fold。測試資料則使用完整訓練資料建立 mapping 後進行編碼。

此外，編碼中加入 smoothing 機制，以減少稀有類別造成的過度擬合。當某類別出現次數很少時，其 encoded value 會被拉近整體平均值；隨著樣本數增加，才會更相信該類別本身的 target 平均值。

以下為程式碼：

```
def make_oof_target_encoding(train, test, features, n_splits=5, smoothing=10, seed=42):
    y = train[TARGET].values
    global_mean = train[TARGET].mean()
    train_encoded = pd.DataFrame(index=train.index)
    test_encoded = pd.DataFrame(index=test.index)
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=seed)

    for feature in features:
        encoded_col = f"{feature}_te"
        train_encoded[encoded_col] = np.nan

        for train_idx, valid_idx in skf.split(train, y):
            fold_train = train.iloc[train_idx]
            fold_valid = train.iloc[valid_idx]
            mapping = smooth_target_mean(
                fold_train[feature],
                fold_train[TARGET],
                global_mean=fold_train[TARGET].mean(),
                smoothing=smoothing,
            )
            train_encoded.loc[fold_valid.index, encoded_col] = (
                fold_valid[feature].map(mapping).fillna(global_mean)
            )

        full_mapping = smooth_target_mean(
            train[feature],
            train[TARGET],
            global_mean=global_mean,
            smoothing=smoothing,
        )
        test_encoded[encoded_col] = test[feature].map(full_mapping).fillna(global_mean)

    return train_encoded, test_encoded
```

(三) One-hot Encoding

One-hot encoding 將每個類別值轉為獨立的二元特徵欄位。相較於 target encoding, 此方法不會洩漏目標資訊, 但會大幅增加特徵維度, 且對高基數(high cardinality)欄位效果較差。本作業於 v1.1 版本中使用此方法與 target encoding 進行比較。

以下為程式碼:

```
model = make_pipeline(  
    OneHotEncoder(handle_unknown="ignore"),  
    LogisticRegression(max_iter=2000, class_weight="balanced", random_state=args.seed),  
)
```

二、模型訓練流程

本作業的通用訓練流程如下:

- 讀取訓練資料與測試資料
- 選取九個類別型特徵作為輸入欄位
- 依版本選擇 target encoding 或 one-hot encoding 進行特徵轉換
- 以 5-fold Stratified Cross-Validation 評估 AUC 作為模型選擇依據
- 使用完整訓練資料重新訓練最終模型
- 對測試資料輸出 ACTION = 1 的預測機率, 產生 Kaggle submission 檔案

三、模型評估

本作業以 Kaggle leaderboard 的 Public Score 與 Private Score (AUC) 作為最終評估指標。在提交前, 以 5-fold Stratified Cross-Validation 的 AUC 均值作為本地評估依據, 用於比較不同模型與參數設定的相對優劣, 並以此決定是否調整參數。

以下為各版本的 CV AUC 與 Kaggle Score 彙整:

版本	模型	Encoding	CV AUC	Public Score	Private Score
v1	Logistic Regression	Target	0.85537	0.86635	0.87230
v1.1	Logistic Regression	One-hot	0.86658	0.88165	0.88803
v2	Random Forest	Target	0.86861	0.86635	0.87703
v2.1	Random Forest	Target	0.86934	0.86755	0.87416

v3	Gradient Boosting	Target	0.86332	0.87009	0.87542
v3.1	Gradient Boosting	Target	0.86385	0.87069	0.87545

整體而言, v1.1 (One-hot + Logistic Regression) 在 Public 與 Private Score 均表現最佳, 顯示對此資料集而言, one-hot encoding 能提供更有利於線性模型的特徵表示。Gradient Boosting 系列 (v3、v3.1) 的 Public Score 雖高於 Random Forest, 但 Private Score 差距不大, 顯示整體泛化能力相近。

四、模型改進流程

本作業共提交六個版本, 依序從簡單線性模型出發, 逐步改用非線性樹模型, 並針對各模型進行參數調整。

v1: Target Encoding + Logistic Regression (Baseline)

第一個版本以 target encoding 搭配 Logistic Regression 作為 baseline, 目的是觀察簡單線性模型在此任務上的表現上限。由於資料存在類別不平衡, 設定 `class_weight='balanced'`。

參數	設定值
max_iter	2000
class_weight	balanced

Kaggle Public Score: 0.86635 | Private Score: 0.87230

v1.1: One-hot Encoding + Logistic Regression

保留 Logistic Regression, 僅將 target encoding 改為 one-hot encoding, 模型參數與 v1 相同。此實驗旨在單純比較特徵表示方式對同一模型的影響。

結果: Public Score 從 0.86635 提升至 0.88165, Private Score 從 0.87230 提升至 0.88803, 為所有版本中最高分。推測原因為 one-hot encoding 保留了類別間的獨立性, 不引入 target 統計量的偏差, 對線性模型更為友好。

Kaggle Public Score: 0.88165 | Private Score: 0.88803

v2: Target Encoding + Random Forest

改用 Random Forest 測試非線性模型的表現。員工權限判斷可能涉及角色、主管、部門及資源之間的複雜組合關係，樹模型理論上能更好地捕捉此類非線性交互效果。

參數	設定值
n_estimators	500
max_depth	12
min_samples_leaf	10
class_weight	balanced_subsample
n_jobs	-1

結果: Public Score 與 v1 相同 (0.86635), Private Score 略有提升 (0.87703)。樹模型並未明顯優於線性模型，可能因為 target encoding 特徵的非線性資訊在此參數設定下尚未充分利用。

Kaggle Public Score: 0.86635 | Private Score: 0.87703

v2.1: Target Encoding + Random Forest (Fine-tune)

在 v2 基礎上提升模型容量: 樹數由 500 增至 800, 最大深度由 12 增至 16, 最小 leaf 樣本數由 10 降至 5。希望讓模型學習更細緻的特徵關係，同時利用更多樹降低預測波動。

參數	設定值
n_estimators	800
max_depth	16
min_samples_leaf	5
class_weight	balanced_subsample
n_jobs	-1

結果: Public Score 小幅提升至 0.86755, 但 Private Score 反而從 0.87703 下降至 0.87416, 顯示模型容量增加後可能有輕微過擬合的情況。

Kaggle Public Score: 0.86755 | Private Score: 0.87416

v3: Target Encoding + Gradient Boosting

改用 Gradient Boosting 作為另一種 tree ensemble 方法。與 Random Forest 並行平均多棵樹不同，Gradient Boosting 逐步加入弱樹修正前一階段的殘差，理論上對複雜特徵交互關係有更強的擬合能力。此版本以保守的學習率與較淺的樹作為起點。

參數	設定值
n_estimators	300
learning_rate	0.05
max_depth	3
min_samples_leaf	20
subsample	0.8

結果:Public Score 提升至 0.87009, Private Score 為 0.87542, 優於 Random Forest 系列, 顯示 Gradient Boosting 的逐步修正機制在此資料上有一定優勢。

Kaggle Public Score:0.87009 | Private Score:0.87542

v3.1: Target Encoding + Gradient Boosting (Fine-tune)

維持 Gradient Boosting 架構, 針對學習動態進行調整:學習率由 0.05 降至 0.03, boosting 輪數由 300 增至 500, 希望透過更小步伐但更多次的修正提升精度。同時將 min_samples_leaf 由 20 降至 10, 測試放寬葉節點限制是否能讓模型學到更細粒度的模式。

參數	設定值
n_estimators	500
learning_rate	0.03
max_depth	3
min_samples_leaf	10
subsample	0.8

結果:Public Score 小幅提升至 0.87069, Private Score 幾乎持平(0.87545)。顯示在此參數空間內, 進一步調整的邊際效益有限, 模型已趨於收斂。

Kaggle Public Score:0.87069 | Private Score:0.87545

五、Submission 截圖紀錄

v1: Target Encoding + Logistic Regression

 submission_1_target_encoding_logistic.csv	0.86635	0.87230	☐
Complete (after deadline) · 21h ago			

v1.1: One-hot Encoding + Logistic Regression

 submission_1.1_one_hot_logistic.csv	0.88165	0.88803	☐
Complete (after deadline) · 9h ago			

v2: Target Encoding + Random Forest

 submission_2_target_encoding_random_fores...	0.86978	0.87703	☐
Complete (after deadline) · 21h ago			

v2.1: Target Encoding + Random Forest (Fine-tune)

 submission_2.1_target_encoding_random_for...	0.86755	0.87416	☐
Complete (after deadline) · 9h ago			

v3: Target Encoding + Gradient Boosting

 submission_3_target_encoding_gradient_boos...	0.87009	0.87542	☐
Complete (after deadline) · 21h ago			

v3.1: Target Encoding + Gradient Boosting (Fine-tune)

 submission_3.1_tuned_gradient_boosting.csv	0.87069	0.87545	☐
Complete (after deadline) · 9h ago			